Start coding or generate with AI.

```
!pip install -q kagglehub

import kagglehub
import tensorflow as tf
import os, shutil
import numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt
```

```
path = kagglehub.dataset_download("kaanerkez/waste-classfication-dataset")

base_path = os.path.join(path, "balanced_waste_images")

print(os.listdir(base_path))
```

```
Downloading to /root/.cache/kagglehub/datasets/kaanerkez/waste-classfication-dataset/1.archive...
100%|██████████| 56.5M/56.5M [00:00<00:00, 89.2MB/s]Extracting files...

['Keyboard', 'PCB', 'paper', 'metal', 'Battery', 'Microwave', 'glass', 'Mobile', 'Player', 'cardboard', 'Printer', 'Mouse', 'Tel
```

```
dst_path = "/content/waste_dataset"

shutil.copytree(base_path, dst_path, dirs_exist_ok=True)

base_path = dst_path

print("Dataset ready at:", base_path)
```

```
Dataset ready at: /content/waste_dataset
```

```
train_dir = "/content/dataset/train"
test_dir = "/content/dataset/test"

os.makedirs(train_dir, exist_ok=True)
os.makedirs(test_dir, exist_ok=True)

for cls in os.listdir(base_path):
    cls_path = os.path.join(base_path, cls)

    if os.path.isdir(cls_path):
        images = os.listdir(cls_path)
        train_imgs, test_imgs = train_test_split(images, test_size=0.2, random_state=42)

        os.makedirs(os.path.join(train_dir, cls), exist_ok=True)
        os.makedirs(os.path.join(test_dir, cls), exist_ok=True)

        for img in train_imgs:
            shutil.copy(os.path.join(cls_path, img),
                        os.path.join(train_dir, cls, img))

        for img in test_imgs:
            shutil.copy(os.path.join(cls_path, img),
                        os.path.join(test_dir, cls, img))
```

```
img_size = (224, 224)
batch_size = 32

train_data = tf.keras.preprocessing.image_dataset_from_directory(
    train_dir,
    image_size=img_size,
    batch_size=batch_size
)

test_data = tf.keras.preprocessing.image_dataset_from_directory(
    test_dir,
    image_size=img_size,
    batch_size=batch_size
)
```

```
class_names = train_data.class_names
print(class_names)
```

```
Found 5423 files belonging to 17 classes.
Found 1359 files belonging to 17 classes.
['Battery', 'Keyboard', 'Microwave', 'Mobile', 'Mouse', 'PCB', 'Player', 'Printer', 'Television', 'Washing Machine', 'cardboard'
```

```
AUTOTUNE = tf.data.AUTOTUNE

train_data = train_data.cache().shuffle(1000).prefetch(AUTOTUNE)
test_data = test_data.cache().prefetch(AUTOTUNE)
```

```
base_model = tf.keras.applications.MobileNetV2(
    input_shape=(224,224,3),
    include_top=False,
    weights='imagenet'
)

base_model.trainable = False

model = tf.keras.Sequential([
    tf.keras.layers.Rescaling(1./255),

    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.2),
    tf.keras.layers.RandomZoom(0.2),

    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),

    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.5),

    tf.keras.layers.Dense(len(class_names), activation='softmax')
])
```

```
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_orde
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
```

```
callbacks = [
    tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=3,
        restore_best_weights=True
    ),
    tf.keras.callbacks.ReduceLROnPlateau(
        monitor='val_loss',
        factor=0.3,
        patience=2
    ),
    tf.keras.callbacks.ModelCheckpoint(
        "best_model.h5",
        save_best_only=True
    )
]
```

```
history = model.fit(
    train_data,
    validation_data=test_data,
    epochs=10,
    callbacks=callbacks
)
```

```
Epoch 1/10
170/170 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.4337 - loss: 2.0487WARNING:absl:You are saving your model as an HDF5 file
170/170 ━━━━━━━━━━━━━━━━━━━━ 375s 2s/step - accuracy: 0.5689 - loss: 1.4765 - val_accuracy: 0.8521 - val_loss: 0.4811 - learning
```

```
Epoch 2/10
170/170 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.6961 - loss: 0.9551WARNING:absl:You are saving your model as an HDF5 file
170/170 ━━━━━━━━━━━━━━━━━━━━ 332s 2s/step - accuracy: 0.7168 - loss: 0.8826 - val_accuracy: 0.8771 - val_loss: 0.3860 - learning
Epoch 3/10
170/170 ━━━━━━━━━━━━━━━━━━━━ 0s 2s/step - accuracy: 0.7482 - loss: 0.7777WARNING:absl:You are saving your model as an HDF5 file
170/170 ━━━━━━━━━━━━━━━━━━━━ 330s 2s/step - accuracy: 0.7575 - loss: 0.7600 - val_accuracy: 0.8911 - val_loss: 0.3197 - learning
Epoch 4/10
170/170 ━━━━━━━━━━━━━━━━━━━━ 329s 2s/step - accuracy: 0.7894 - loss: 0.6508 - val_accuracy: 0.8823 - val_loss: 0.3420 - learning
Epoch 5/10
157/170 ━━━━━━━━━━━━━━━━━━━━ 20s 2s/step - accuracy: 0.7968 - loss: 0.6095
```

```
base_model.trainable = True

for layer in base_model.layers[:-20]:
    layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

history_fine = model.fit(
    train_data,
    validation_data=test_data,
    epochs=5,
    callbacks=callbacks
)
```

```
loss, acc = model.evaluate(test_data)
print("Final Accuracy:", acc)
```

```
from google.colab import files

def predict_upload():
    uploaded = files.upload()

    for img_name in uploaded.keys():
        img = image.load_img(img_name, target_size=(224,224))
        img_array = image.img_to_array(img)
        img_array = np.expand_dims(img_array, axis=0)

        pred = model.predict(img_array)

        plt.imshow(img)
        plt.title("Predicted: " + class_names[np.argmax(pred)])
        plt.axis("off")
        plt.show()

predict_upload()
```

Start coding or generate with AI.